

Cartographer-SLAM Mapping Algorithm

1. Course Content

- 1. Learn the Cartographer SLAM mapping algorithm for robot SLAM mapping functionality

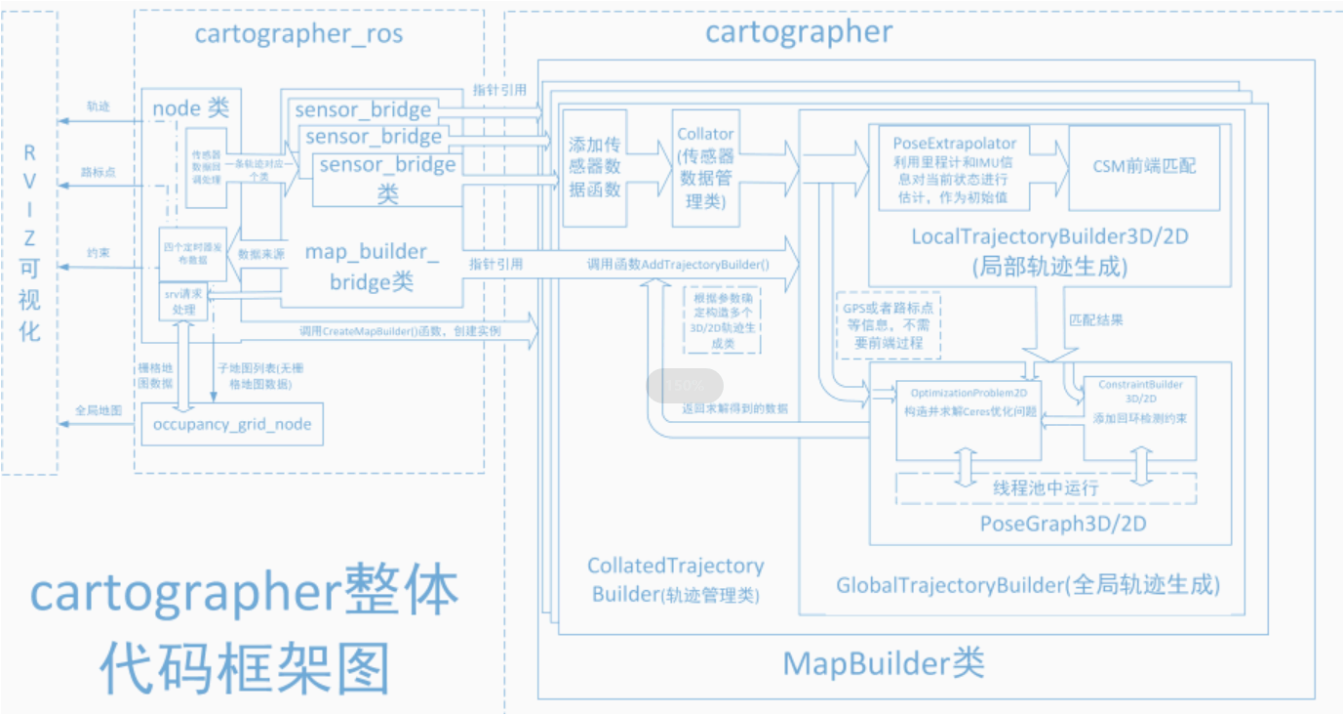
After running the sample program, use the keyboard or controller to move the robot to complete map construction and save the map.

2. Introduction to Cartographer

2.1 Introduction

Cartographer is a 2D and 3D SLAM (simultaneous localization and mapping) library open-sourced by Google that supports the ROS system. It is a mapping algorithm based on graph optimization (multi-threaded backend optimization, problem optimization built with Ceres). It can combine data from multiple sensors (e.g., LIDAR, IMU, and cameras) to simultaneously calculate the sensor's position and map the environment around the sensor.

The source code of Cartographer is mainly consists of three parts: cartographer, cartographer_ros, and ceres-solver (backend optimization).



Cartographer adopts the mainstream SLAM framework, i.e., the three-stage process of feature extraction, loop closure detection, and backend optimization. A certain number of LaserScans form a submap, and a series of submaps constitute the global map. The cumulative error during the short process of constructing a submap with LaserScans is small, but during the long process of building the global map with submaps, there will be significant cumulative error. Therefore, loop closure detection is needed to correct the positions of these submaps. The basic unit of loop closure detection is the submap, and loop closure detection uses a scan_match strategy. The key content of Cartographer is the creation of submap submaps by fusing multi-sensor data (odometry, IMU, LaserScan, etc.) and the implementation of the scan_match strategy for loop closure detection.

- cartographer_ros

cartographer_ros runs under ROS and can receive various sensor data in the form of ROS messages. After processing, it publishes them as messages for debugging and visualization.

2.2 Related Resources

[GitHub Repository](#).

[Official Documentation](#)

3. Preparation

3.1 Content Description

This course uses jetson orin NX as an example. For Raspberry Pi and Jetson-nano motherboards, you need to open a terminal on the host machine, enter the command to access the docker container, and then enter the commands mentioned in this course within the docker container terminal. For the tutorial on accessing the docker container from the host machine, refer to the [0. Instructions and Installation Steps] section of this product tutorial, specifically [Entering the Car Docker (For Jetson-Nano and Raspberry Pi 5 Users)]. For orin and NX motherboards, simply open a terminal and enter the commands mentioned in this course.

3.2 Start the Agent

Note: All test cases must start the docker agent first. If already started, no need to restart.

Enter the command in the robot terminal:

```
sh start_agent.sh
```

The terminal prints the following information, indicating a successful connection.

```

jetson@yahboom:~$ ./open_agent.sh
[1749538760.063253] info | TermiosAgentLinux.cpp | init | running... | fd: 3
[1749538760.064012] info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4
[1749538760.941420] info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81
[1749538760.941537] info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0
[1749538760.979971] info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)
[1749538760.983835] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)
[1749538760.988244] info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)
[1749538760.993117] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)
[1749538760.997675] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)
[1749538761.001121] info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)
[1749538761.007277] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)
[1749538761.010634] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)
[1749538761.014296] info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)
[1749538761.020171] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)
[1749538761.023939] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)
[1749538761.029173] info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)
[1749538761.034377] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)
[1749538761.038946] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)
[1749538761.042215] info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)
[1749538761.048422] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)
[1749538761.051660] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)
[1749538761.057494] info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)
[1749538761.062183] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)
[1749538761.066842] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)
[1749538761.071217] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)

```

4. Running the Case

4.1 Mapping Process

Note:

- When mapping, slower speeds yield better results (especially rotation speed). High speed will result in poor quality.
- Jetson Nano and Raspberry Pi series controllers need to enter the docker container first (see [Docker Course Chapter - Entering the Robot Docker Container]).

Start the underlying sensor command on the robot terminal:

```
ros2 launch slam_mapping bringup.launch.py
```

Then start the mapping command:

```
ros2 launch slam_mapping cartographer.launch.py
```

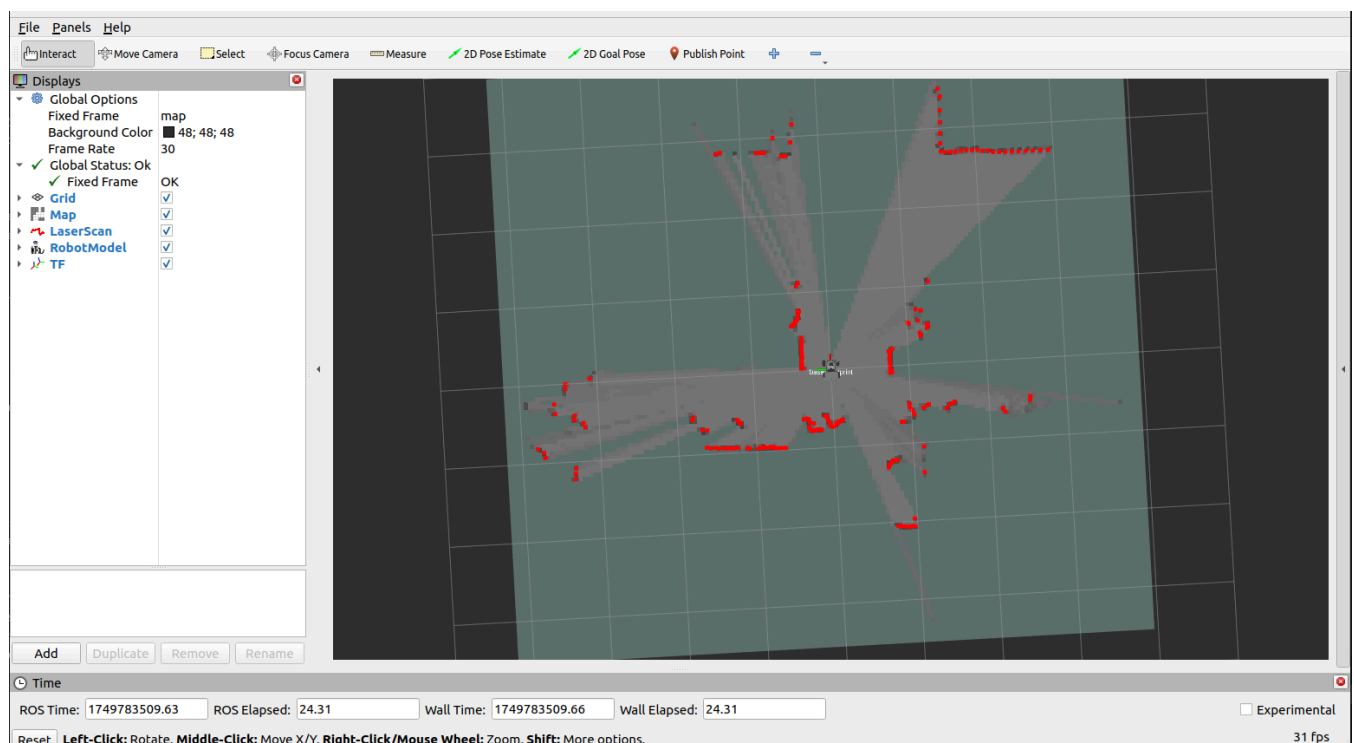
The rviz visualization can be started on either the robot side or the virtual machine side. **Choose one** method. Do not start both on the virtual machine and the robot simultaneously.

Using the virtual machine as an example, open a terminal and start the rviz visualization interface:

```
ros2 launch slam_view slam_view.launch.py
```

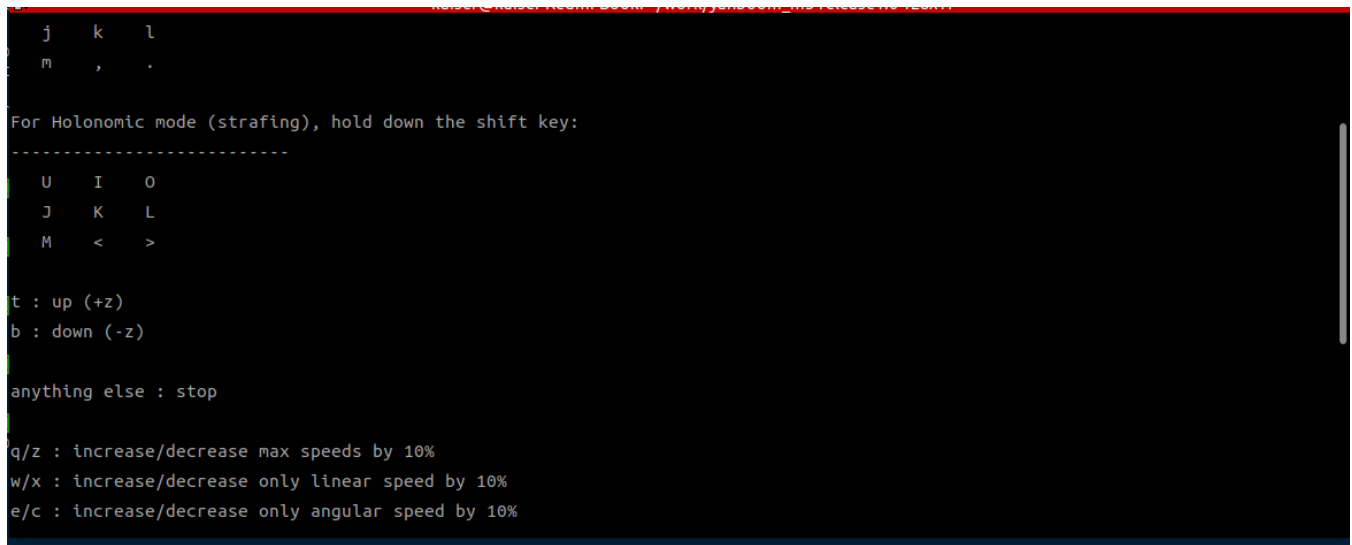
Command to start the rviz visualization interface on the robot side:

```
ros2 launch slam_mapping slam_view.launch.py
```

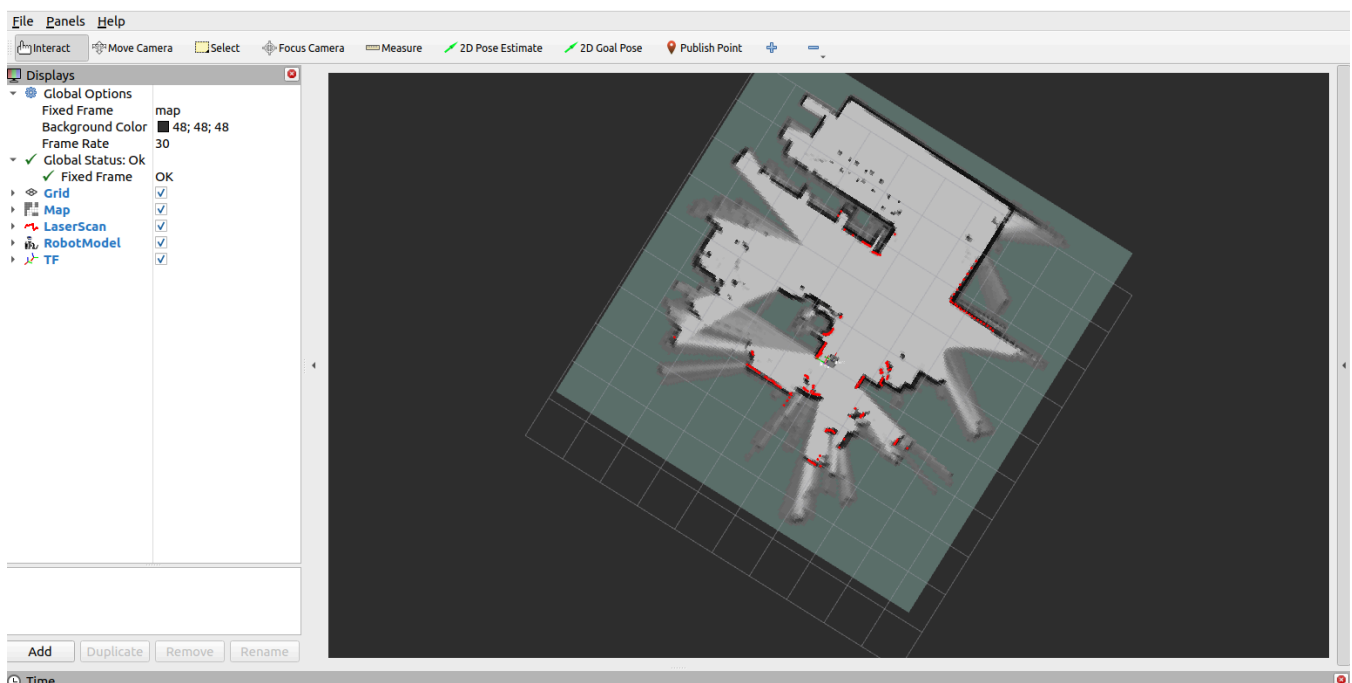


Open another terminal on the virtual machine to start the keyboard control node (a game controller can also be used):

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```



Click the terminal window with the mouse, press z to reduce speed appropriately, and press I, <, J, L to control the robot forward, backward, left turn, and right turn respectively. Control the robot to move slowly to complete mapping.



4.2 Saving the Map

Open a new terminal on the robot side to save the map:

```
ros2 launch slam_mapping save_map.launch.py
```

The terminal prompt **Map saved successfully** indicates the map was saved successfully.

```
jetson@yahboom: ~  
jetson@yahboom: ~  
jetson@yahboom:~$ ros2 launch slam_mapping save_map.launch.py  
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-06-13-15-21-29-707592-yahboom-274913  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [map_saver_cli-1]: process started with pid [274915]  
[map_saver_cli-1] [INFO] [1749799289.958644288] [map_saver]:  
map_saver lifecycle node launched.  
[map_saver_cli-1] Waiting on external lifecycle transitions to activate  
[map_saver_cli-1] See https://design.ros2.org/articles/node_lifecycle.html for more information.  
[map_saver_cli-1] [INFO] [1749799289.958949633] [map_saver]: Creating  
[map_saver_cli-1] [INFO] [1749799289.959153218] [map_saver]: Configuring  
[map_saver_cli-1] [INFO] [1749799289.962640750] [map_saver]: Saving map from 'map' topic to '/home/jetson/M3Pro_ws/install/M3Pro_navigation/share/M3Pro_navigation/map/yahboom_map' file  
[map_saver_cli-1] [WARN] [1749799290.088323133] [map_io]: Image format unspecified. Setting it to: pgm  
[map_saver_cli-1] [INFO] [1749799290.088477342] [map_io]: Received a 237 X 267 map @ 0.05 m/pix  
[map_saver_cli-1] [INFO] [1749799290.128675762] [map_io]: Writing map occupancy data to /home/jetson/M3Pro_ws/install/M3Pro_navigation/share/M3Pro_navigation/map/yahboom_map.pgm  
[map_saver_cli-1] [INFO] [1749799290.130244184] [map_io]: Writing map metadata to /home/jetson/M3Pro_ws/install/M3Pro_navigation/share/M3Pro_navigation/map/yahboom_map.yaml  
[map_saver_cli-1] [INFO] [1749799290.130454681] [map_io]: Map saved  
[map_saver_cli-1] [INFO] [1749799290.130478329] [map_saver]: Map saved successfully  
[map_saver_cli-1] [INFO] [1749799290.133247907] [map_saver]: Destroying  
[INFO] [map_saver_cli-1]: process has finished cleanly [pid 274915]  
jetson@yahboom:~$
```

The map save path is as follows:

```
$HOME/M3Pro_ws/install/M3Pro_navigation/share/M3Pro_navigation/map
```

One pgm image and one yaml file yahboom_map.yaml

```
image: yahboom_map.pgm  
mode: trinary  
resolution: 0.05  
origin: [-10, -10, 0]  
negate: 0  
occupied_thresh: 0.65  
free_thresh: 0.25
```

Parameter explanation:

- image: The path to the map file, can be an absolute path or a relative path.
- mode: This attribute can be one of trinary, scale, or raw. trinary is the default mode.
- resolution: The resolution of the map, meters/pixel.
- origin: The 2D pose (x, y, yaw) of the lower-left corner of the map. yaw is rotated counterclockwise (yaw=0 means no rotation). Many parts of the current system may ignore the yaw value.
- negate: Whether to invert the meaning of white/black, free/occupied (threshold interpretation is not affected).
- occupied_thresh: Pixels with an occupancy probability greater than this threshold are considered fully occupied.
- free_thresh: Pixels with an occupancy probability less than this threshold are considered completely free.

4.3 Saving pbstream Format Map

The pbstream format map file is used for relocation navigation and will be explained in subsequent chapters.

After map construction is complete, open a new terminal and enter the command to finish the map trajectory:

```
ros2 service call /finish_trajectory cartographer_ros_msgs/srv/FinishTrajectory "{trajectory_id: 0}"
```

Then open another terminal to save the pbstream format map file:

For jetson orin nano and jetson orin NX hosts, the save command is:

```
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState "{filename: '/home/jetson/yahboom_map.pbstream'}"
```

For jetson orin nano and Raspberry Pi hosts, the save command is:

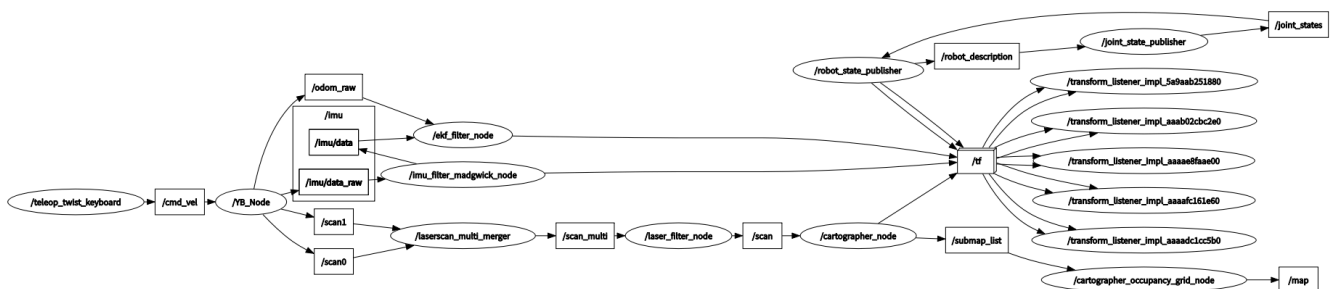
First enter docker:

```
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState "{filename: '/root/yahboom_map.pbstream'}"
```

5. Node Analysis

5.1 Display Node Computation Graph

```
ros2 run rqt_graph rqt_graph
```

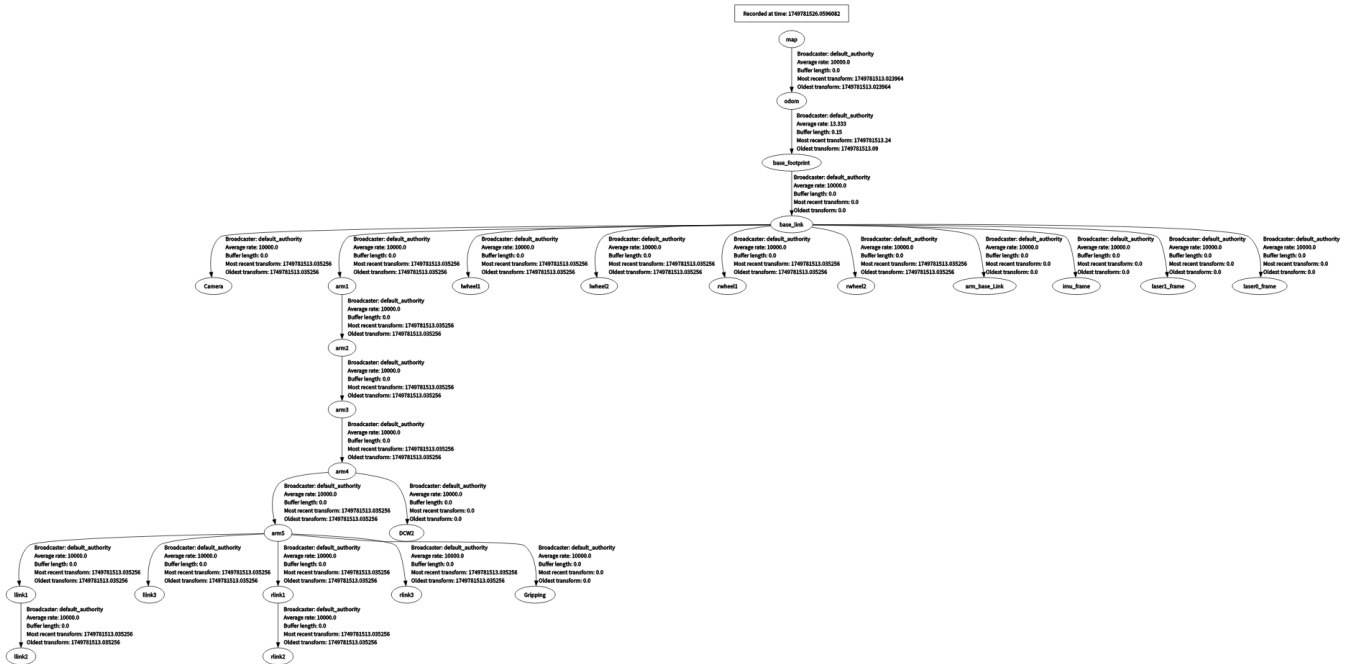


5.2 TF Transform

Run on the virtual machine terminal:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

The image is too large. The original image can be viewed in this course's folder.



5.3 Cartographer Node Details

```
ros2 node info /slam_gmapping
```

Enter the above command in the terminal to view the subscription and publication topics related to the gmapping node.

```
/cartographer_node
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /scan: sensor_msgs/msg/LaserScan
Publishers:
  /constraint_list: visualization_msgs/msg/MarkerArray
  /landmark_poses_list: visualization_msgs/msg/MarkerArray
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /scan_matched_points2: sensor_msgs/msg/PointCloud2
  /submap_list: cartographer_ros_msgs/msg/SubmapList
  /tf: tf2_msgs/msg/TFMessage
  /trajectory_node_list: visualization_msgs/msg/MarkerArray
Service Servers:
  /cartographer_node/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /cartographer_node/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /cartographer_node/get_parameters: rcl_interfaces/srv/GetParameters
  /cartographer_node/list_parameters: rcl_interfaces/srv/ListParameters
  /cartographer_node/set_parameters: rcl_interfaces/srv/SetParameters
  /cartographer_node/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  /finish_trajectory: cartographer_ros_msgs/srv/FinishTrajectory
  /get_trajectory_states: cartographer_ros_msgs/srv/GetTrajectoryStates
  /read_metrics: cartographer_ros_msgs/srv/ReadMetrics
  /start_trajectory: cartographer_ros_msgs/srv/StartTrajectory
  /submap_query: cartographer_ros_msgs/srv/SubmapQuery
  /tf2_frames: tf2_msgs/srv/FrameGraph
```

```
/trajectory_query: cartographer_ros_msgs/srv/TrajectoryQuery  
/write_state: cartographer_ros_msgs/srv/writeState  
Service Clients:
```

Action Servers:

Action Clients: